

Abstract class in Java

A class which is declared with the abstract keyword is known as an abstract class in [Java](#)

It can have abstract and non-abstract methods (method with the body).

Abstraction in Java

Abstraction is a process of hiding the implementation details and showing only functionality to the user.

Interface in Java

An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.

The interface in Java is *a mechanism to achieve [abstraction](#)*.

There can be only abstract methods in the Java interface, not method body.

It is used to achieve abstraction and multiple [inheritance in Java](#).

In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method body.

Java Interface also **represents the IS-A relationship**.

It cannot be instantiated just like the abstract class.

Why use Java interface?

There are mainly three reasons to use interface. They are given below.

- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritance.
- It can be used to achieve loose coupling.

How to declare an interface?

An interface is declared by using the interface keyword.

It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default.

A class that implements an interface must implement all the methods declared in the interface.

Syntax:

1. **interface** <interface_name>{
- 2.
3. // declare constant fields
4. // declare methods tha

How to declare an interface?

An interface is declared by using the interface keyword.

It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default.

A class that implements an interface must implement all the methods declared in the interface.

Syntax:

1. **interface** <interface_name>{
- 2.
3. // declare constant fields
4. // declare methods tha

Java Interface Example

In this example, the Printable interface has only one method, and its implementation is provided in the A6 class.

1. **interface** printable{
2. **void** print();
3. }

Nilam Suthar
Pgdca dept

```

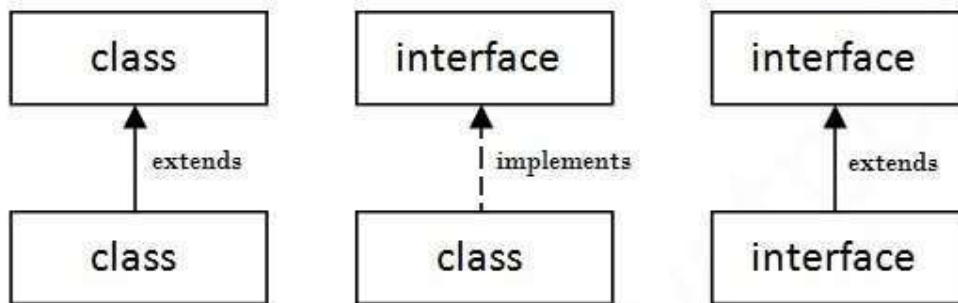
4. class A6 implements printable{
5. public void print(){System.out.println("Hello");}
6.
7. public static void main(String args[]){
8. A6 obj = new A6();
9. obj.print();
10. }
11. }
    
```

-
- *//Interface declaration: by first user*
- **interface** Drawable{
- **void** draw();
- }
- *//Implementation: by second user*
- **class** Rectangle **implements** Drawable
- {
- **public void** draw()
- {
- System.out.println("drawing rectangle");
- }
- }
- **class** Circle **implements** Drawable
- {
- **public void** draw()
- {
- System.out.println("drawing circle");
- }
- }
- *//Using interface: by third user*
- **class** TestInterface1
- {
- **public static void** main(String args[])
- {
- Drawable d=**new** Circle();//In real scenario, object is provided by method e.g. *getDrawable()*
- d.draw();

- }}

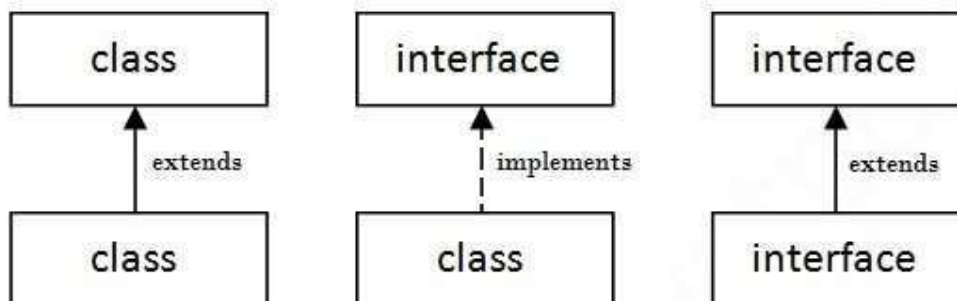
The relationship between classes and interfaces

As shown in the figure given below, a class extends another class, an interface extends another interface, but a **class implements an interface**.



The relationship between classes and interfaces

As shown in the figure given below, a class extends another class, an interface extends another interface, but a **class implements an interface**.



Java Package

Nilam Suthar
Pgdc dept

A **java package** is a group of similar types of classes, interfaces and sub-packages.

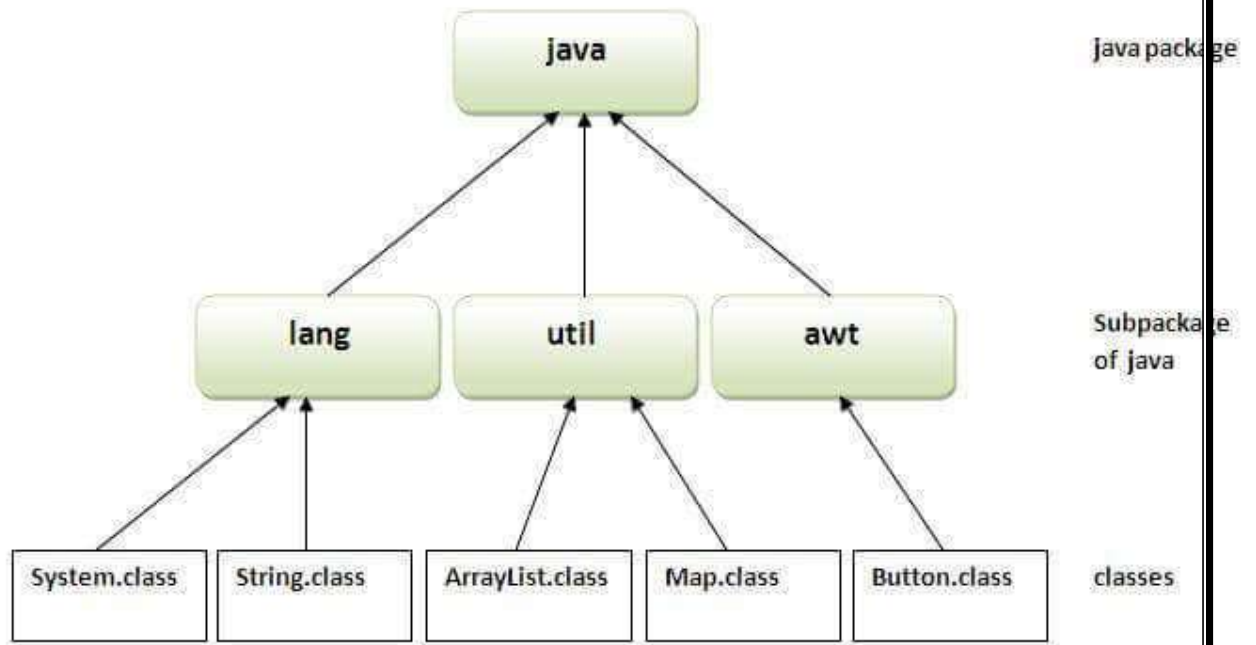
Package in java can be categorized in two form, built-in package and user-defined package.

There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Here, we will have the detailed learning of creating and using user-defined packages.

Advantage of Java Package

- 1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming collision.



Simple example of java package

The **package keyword** is used to create a package in java

1. `//save as Simple.java`
2. `package mypack;`
3. `public class Simple{`
4. `public static void main(String args[]){`
5. `System.out.println("Welcome to package");`
6. `}`
7. `}`

How to compile java package

If you are not using any IDE, you need to follow the **syntax** given below:

How to access package from another package?

There are three ways to access the package from outside the package.

1. `import package.*;`
2. `import package.classname;`
3. fully qualified name.

1) *Using packagename.**

If you use `package.*` then all the classes and interfaces of this package will be accessible but not subpackages.

The `import` keyword is used to make the classes and interface of another package accessible to the current package.

Example of package that import the packagename

//save by A.java

1. *//save by A.java*
2. **package** pack;
3. **public class** A
4. {
5. **public void** msg()
6. {
7. System.out.println("Hello");
8. }
9. }

1. *//save by B.java*
2. **package** mypack;
3. **import** pack.*;
- 4.
5. **class** B
6. {
7. **public static void** main(String args[]){
8. A obj = **new** A();

```
9.    obj.msg();  
10. }  
11.}
```

```
Output:Hello
```

2) Using *package.classname*

If you import `package.classname` then only declared class of this package will be accessible.

Example of package by import `package.classname`

```
1. //save by A.java  
2.  
3. package pack;  
4. public class A  
5. {  
6.     public void msg()  
7. {  
8.     System.out.println("Hello");  
9. }  
10. }
```

```
1. //save by B.java  
2. package mypack;  
3. import pack.A;  
4.  
5. class B  
6. {  
7.     public static void main(String args[])  
8. {  
9.     A obj = new A();  
10.    obj.msg();  
11. }  
12. }
```

```
Output:Hello
```


Package: java.lang

The java.lang package provides classes that are fundamental to the design of the Java programming language.

This is the most important package of java API.

And also the most important class is Object because Object is the parent of all classes.

Every classes and instances in java is a child of Object.

The wrapper classes Boolean, Character, Integer, Long, Float, and Double serve this purpose.

An object of type Double, for example, contains a field whose type is double, representing that value in such a way that a reference to it can be stored in a variable of reference type.

Integer

The Integer class wraps the int primitive data type into an object. This class includes helpful methods in converting value from string to Integer.

Math

The class Math contains methods for performing basic numeric operations such as the elementary exponential, logarithm, square root, and trigonometric functions.

String

Java String class is represented by character strings.

StringBuffer

The StringBuffer is a thread safe, mutable sequence of characters. Its much the same as String however StringBuffer can be modified.

Double

The Double class wraps a value of the primitive type double in an object. An object of type Double contains a single field whose type is double.

Java Stack

The **stack** is a linear data structure that is used to store the collection of objects.

Nilam Suthar
Pgdc dept

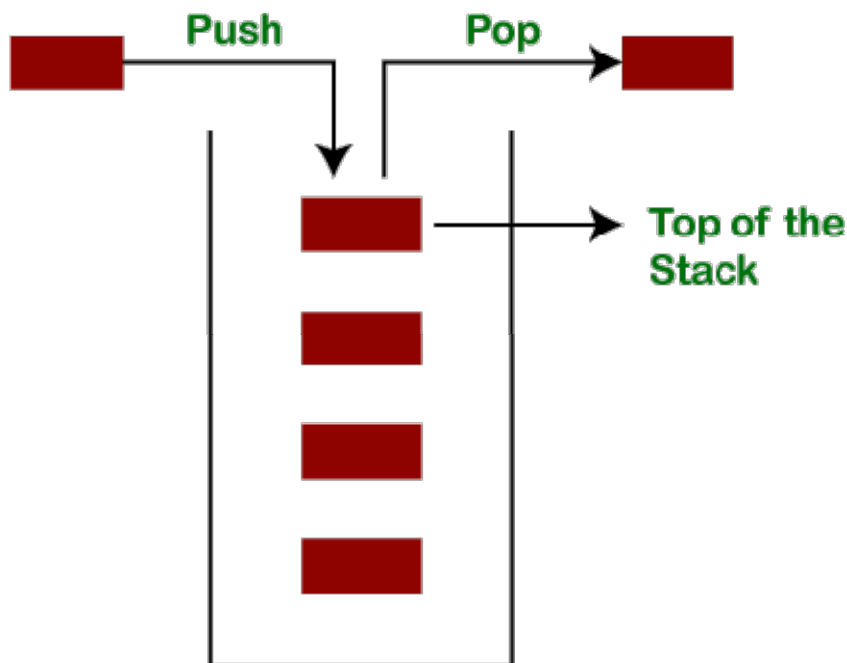
It is based on **Last-In-First-Out** (LIFO). **Java collection** framework provides many interfaces and classes to store the collection of objects.

One of them is the **Stack class** that provides different operations such as push, pop, search, etc.

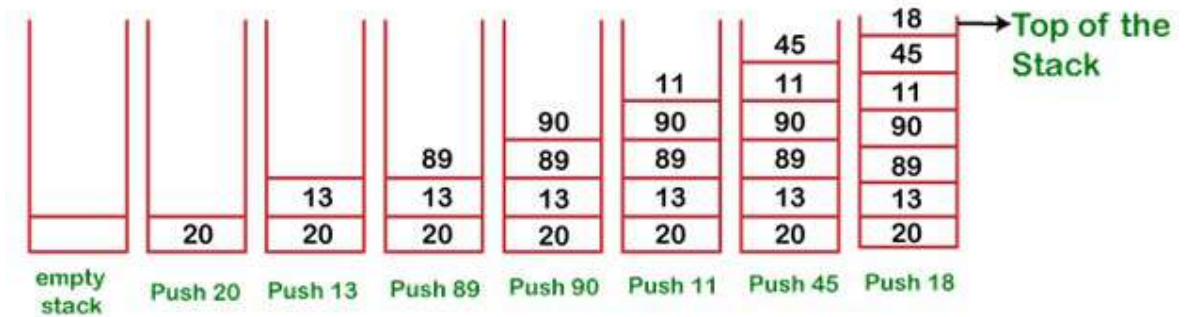
In this section, we will discuss the **Java Stack class**, its **methods**, and **implement** the stack data structure in a **Java program**.

The stack data structure has the two most important operations that are **push** and **pop**.

The push operation inserts an element into the stack and pop operation removes an element from the top of the stack. Let's see how they work on stack.

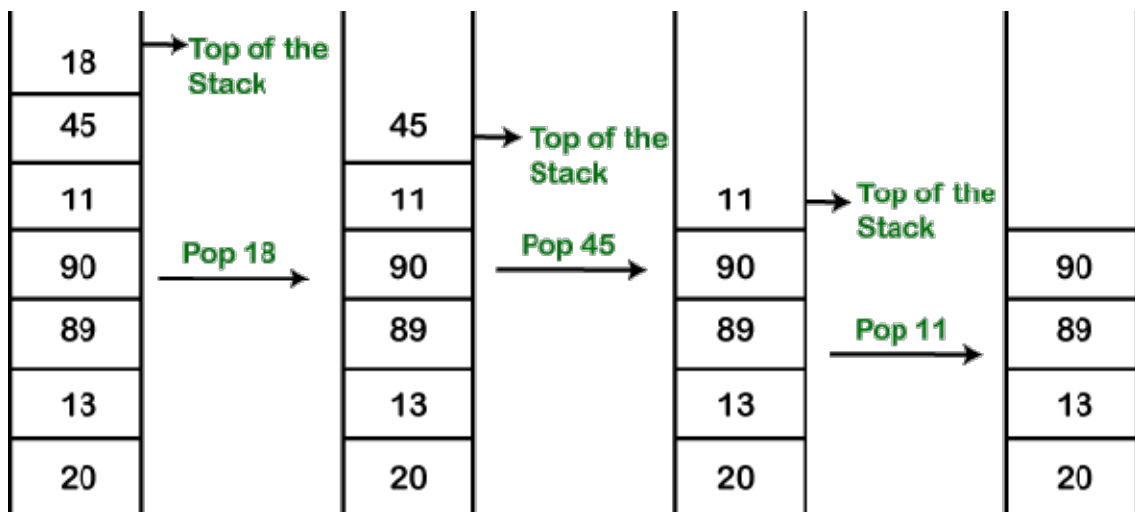


Let's push 20, 13, 89, 90, 11, 45, 18, respectively into the stack.



Push operation

Let's remove (pop) 18, 45, and 11 from the stack.



Pop operation

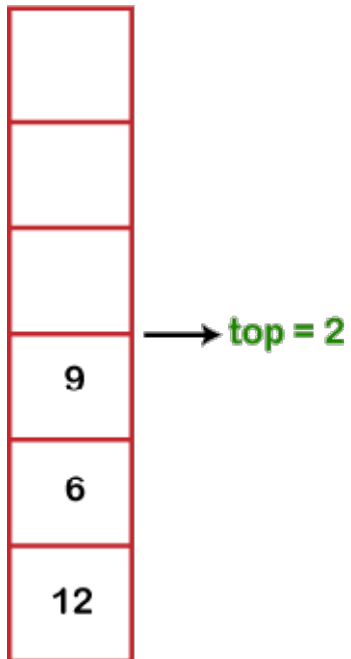
Empty Stack: If the stack has no element is known as an **empty stack**. When the stack is empty the value of the top variable is -1.

Empty Stack

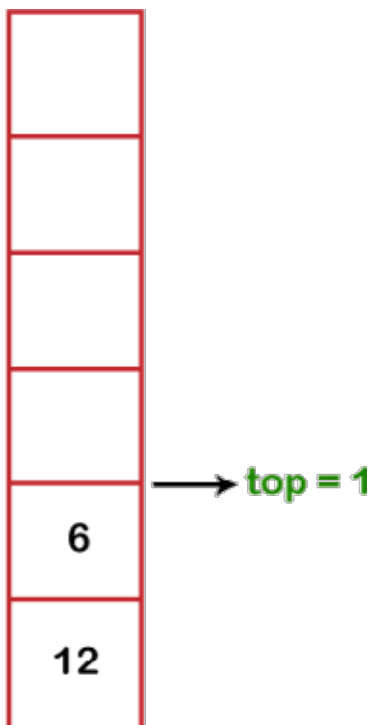


When we push an element into the stack the top is **increased by 1**. In the following figure,

- Push 12, top=0
- Push 6, top=1
- Push 9, top=2



When we pop an element from the stack the value of top is **decreased by 1**. In the following figure, we have popped 9.



The following table shows the different values of the top.

Top value	Meaning
-1	It shows the stack is empty.
0	The stack has only an element.
N-1	The stack is full.
N	The stack is overflow.

Java Stack Class

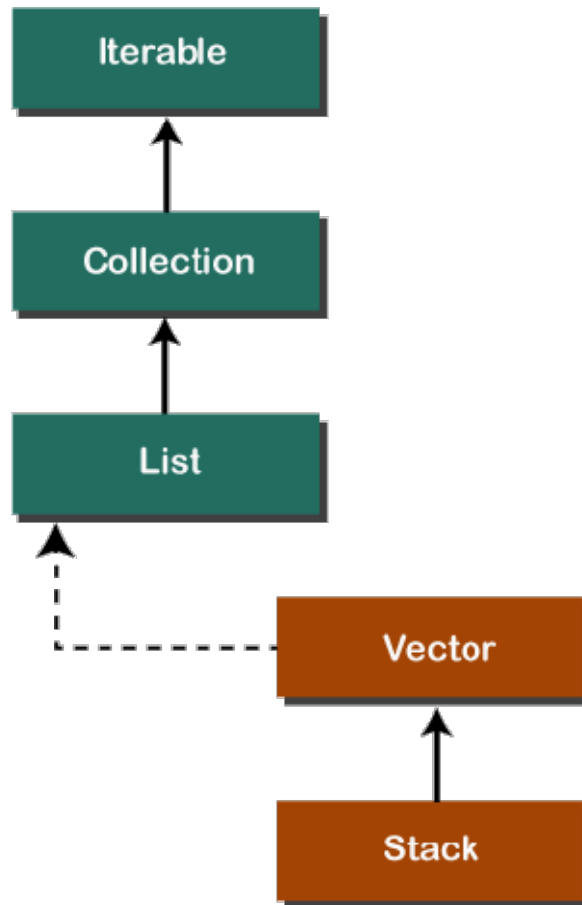
In Java, **Stack** is a class that falls under the Collection framework that extends the **Vector** class.

It also implements interfaces **List, Collection, Iterable, Cloneable, Serializable.**

It represents the LIFO stack of objects.

Before using the Stack class, we must import the `java.util` package.

The stack class arranged in the Collections framework hierarchy, as shown below.



Stack Class Constructor

The Stack class contains only the **default constructor** that creates an empty stack.

1. `public Stack()`

Creating a Stack

If we want to create a stack, first, import the `java.util` package and create an object of the Stack class.

1. `Stack stk = new Stack();`

Or

1. `Stack<type> stk = new Stack<>();`

Where type denotes the type of stack like Integer, String, etc.

Methods of the Stack Class

We can perform push, pop, peek and search operation on the stack. The Java Stack class provides mainly five methods to perform these operations. Along with this, it also provides all the methods of the [Java Vector class](#).

Method	Modifier and Type	Method Description
empty()	boolean	The method checks the stack is empty or not.
push(E item)	E	The method pushes (insert) an element onto the top of the stack.
pop()	E	The method removes an element from the top of the stack and returns the same element as the value of that function.
peek()	E	The method looks at the top element of the stack without removing it.
search(Object o)	int	The method searches the specified object and returns the position of the object.

Stack Class empty() Method

The **empty()** method of the Stack class check the stack is empty or not. If the stack is empty, it returns true, else returns false. We can also use the [isEmpty\(\) method of the Vector class](#).

Syntax

1. **public boolean** empty()

Returns: The method returns true if the stack is empty, else returns false.

In the following example, we have created an instance of the Stack class. After that, we have invoked the empty() method two times. The first time it returns **true** because we have not pushed any element into the stack. After that, we have pushed elements into

the stack. Again we have invoked the `empty()` method that returns **false** because the stack is not empty.

StackEmptyMethodExample.java

```
1. import java.util.Stack;
2. public class StackEmptyMethodExample
3. {
4.     public static void main(String[] args)
5.     {
6.         //creating an instance of Stack class
7.         Stack<Integer> stk= new Stack<>();
8.         // checking stack is empty or not
9.         boolean result = stk.empty();
10.        System.out.println("Is the stack empty? " + result);
11.        // pushing elements into stack
12.        stk.push(78);
13.        stk.push(113);
14.        stk.push(90);
15.        stk.push(120);
16.        //prints elements of the stack
17.        System.out.println("Elements in Stack: " + stk);
18.        result = stk.empty();
19.        System.out.println("Is the stack empty? " + result);
20.    }
21. }
```

Output:

Stack Class push() Method

The method inserts an item onto the top of the stack. It works the same as the method [addElement\(item\) method](#) of the Vector class. It passes a parameter **item** to be pushed into the stack.

Syntax

```
1. public E push(E item)
```

Parameter: An item to be pushed onto the top of the stack.

Returns: The method returns the argument that we have passed as a parameter.

Stack Class pop() Method

The method removes an object at the top of the stack and returns the same object. It throws **EmptyStackException** if the stack is empty.

Syntax

1. **public** E pop()

Returns: It returns an object that is at the top of the stack.

Let's implement the stack in a Java program and perform push and pop operations.

StackPushPopExample.java

1. **import** java.util.*;
2. **public class** StackPushPopExample
3. {
4. **public static void** main(String args[])
5. {
6. *//creating an object of Stack class*
7. Stack <Integer> stk = **new** Stack<>();
8. System.out.println("stack: " + stk);
9. *//pushing elements into the stack*
10. pushelmnt(stk, 20);
11. pushelmnt(stk, 13);
12. pushelmnt(stk, 89);
13. pushelmnt(stk, 90);
14. pushelmnt(stk, 11);
15. pushelmnt(stk, 45);
16. pushelmnt(stk, 18);
17. *//popping elements from the stack*
18. popelmnt(stk);
19. popelmnt(stk);
20. *//throws exception if the stack is empty*

Nilam Suthar
Pgdca dept

```
21. try
22. {
23. popelmnt(stk);
24. }
25. catch (EmptyStackException e)
26. {
27. System.out.println("empty stack");
28. }
29. }
30. //performing push operation
31. static void pushelmnt(Stack stk, int x)
32. {
33. //invoking push() method
34. stk.push(new Integer(x));
35. System.out.println("push -> " + x);
36. //prints modified stack
37. System.out.println("stack: " + stk);
38. }
39. //performing pop operation
40. static void popelmnt(Stack stk)
41. {
42. System.out.print("pop -> ");
43. //invoking pop() method
44. Integer x = (Integer) stk.pop();
45. System.out.println(x);
46. //prints modified stack
47. System.out.println("stack: " + stk);
48. }
49. }
```

Output:

```
stack: []
push -> 20
stack: [20]
push -> 13
stack: [20, 13]
push -> 89
stack: [20, 13, 89]
push -> 90
```

Nilam Suthar
Pgdca dept

```

stack: [20, 13, 89, 90]
push -> 11
stack: [20, 13, 89, 90, 11]
push -> 45
stack: [20, 13, 89, 90, 11, 45]
push -> 18
stack: [20, 13, 89, 90, 11, 45, 18]
pop -> 18
stack: [20, 13, 89, 90, 11, 45]
pop -> 45
stack: [20, 13, 89, 90, 11]
pop -> 11
stack: [20, 13, 89, 90]

```

Java Vector

Vector is like the *dynamic array* which can grow or shrink its size.

Unlike array, we can store n-number of elements in it as there is no size limit.

It is found in the `java.util` package and implements the *List* interface, so we can use all the methods of List interface here.

The Iterators returned by the Vector class are *fail-fast*

t. n.

It is similar to the ArrayList, but with two differences-

Java Vector class Declaration

1. **public class** Vector<E>
2. **extends** Object<E>
3. **implements** List<E>, Cloneable, Serializable

Java Vector Constructors

Vector class supports four types of constructors. These are given below:

SN	Constructor	Description
1)	vector()	It constructs an empty vector with the default size as 10.

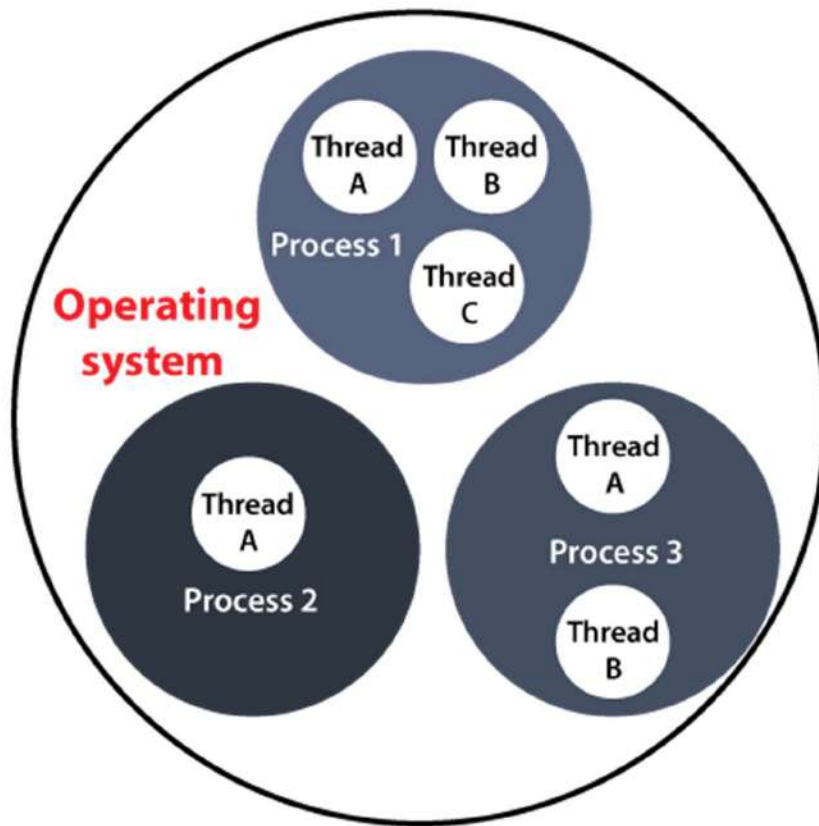
2)	<code>vector(int initialCapacity)</code>	It constructs an empty vector with the specified initial capacity and with its capacity increment equal to zero.
3)	<code>vector(int initialCapacity, int capacityIncrement)</code>	It constructs an empty vector with the specified initial capacity and capacity increment.
4)	<code>Vector(Collection<? extends E> c)</code>	It constructs a vector that contains the elements of a collection c.

Java Vector Methods

The following are the list of Vector class methods:

SN	Method	Description
1)	<code>add()</code>	It is used to append the specified element in the given vector.
2)	<code>addAll()</code>	It is used to append all of the

thread

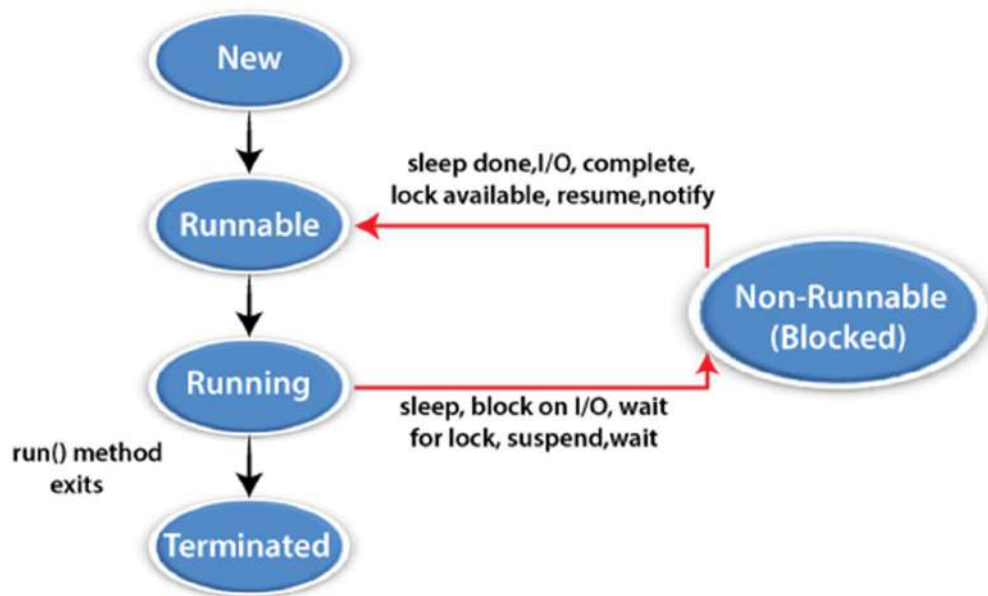


a Thread is a:

- Feature through which we can perform multiple activities within a single process.
- Lightweight process.
- Series of executed statements.
- Nested sequence of method calls.

Thread Model

Just like a process, a thread exists in several states. These states are as follows:



1) New (Ready to run)

A thread is in **New** when it gets CPU time.

2) Running

A thread is in a **Running** state when it is under execution.

3) Suspended

A thread is in the **Suspended** state when it is temporarily inactive or under execution.

4) Blocked

A thread is in the **Blocked** state when it is waiting for resources.

5) Terminated

A thread comes in this state when at any given time, it halts its execution immediately.

Creating Thread

A thread is created either by "creating or implementing" the **Runnable Interface** or by extending the **Thread class**. These are the only two ways through which we can create a thread.

Let's dive into details of both these way of creating a thread:

Thread Class

A **Thread** class has several methods and constructors which allow us to perform various operations on a thread. The Thread class extends the **Object** class. The **Object** class implements the **Runnable** interface. The thread class has the following constructors that are used to perform various operations.

- **Thread()**
- **Thread(Runnable, String name)**
- **Thread(Runnable target)**
- **Thread(ThreadGroup group, Runnable target, String name)**
- **Thread(ThreadGroup group, Runnable target)**
- **Thread(ThreadGroup group, String name)**
- **Thread(ThreadGroup group, Runnable target, String name, long stackSize)**

Runnable Interface(run() method)

The Runnable interface is required to be implemented by that class whose instances are intended to be executed by a thread.

The runnable interface gives us the **run()** method to perform an action for the thread.

start() method

The method is used for starting a thread that we have newly created.

It starts a new thread with a new callstack.

After executing the **start()** method, the thread changes the state from New to Runnable.

It executes the **run() method** when the thread gets the correct time to execute it.

ThreadExample1.java

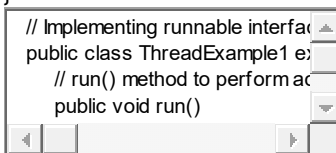
1. `// Implementing runnable interface by extending Thread class`
2. `public class ThreadExample1 extends Thread {`
3. `// run() method to perform action for thread.`

Nilam Suthar
Pgdca dept


```

4.  public void run()
5.  {
6.      int a= 10;
7.      int b=12;
8.      int result = a+b;
9.      System.out.println("Thread started running..");
10.     System.out.println("Sum of two numbers is: " + result);
11. }
12. public static void main( String args[] )
13. {
14.     // Creating instance of the class extend Thread class
15.     ThreadExample1 t1 = new ThreadExample1();
16.     //calling start method to execute the run() method of the Thread class
17.     t1.start();
18. }
19. }

```

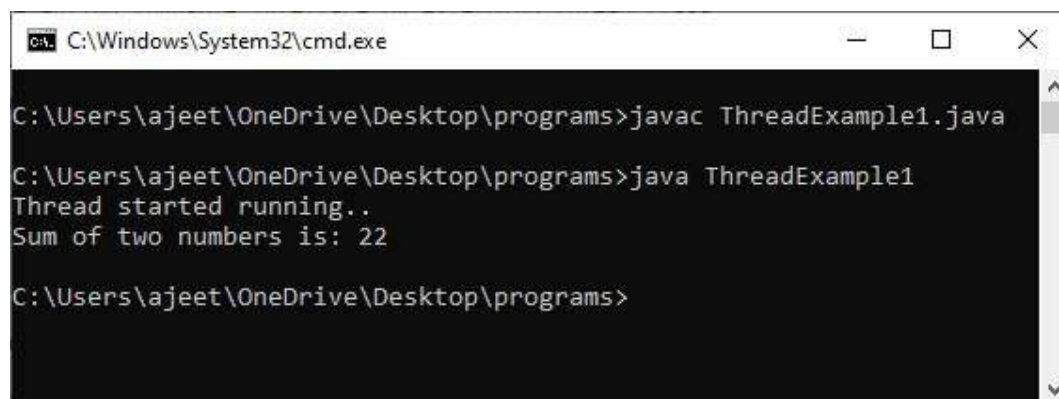


```

// Implementing runnable interface
public class ThreadExample1 extends Thread {
    // run() method to perform the task
    public void run() {

```

Output:



```

C:\Windows\System32\cmd.exe

C:\Users\ajeet\OneDrive\Desktop\programs>javac ThreadExample1.java

C:\Users\ajeet\OneDrive\Desktop\programs>java ThreadExample1
Thread started running..
Sum of two numbers is: 22

C:\Users\ajeet\OneDrive\Desktop\programs>

```

Creating thread by implementing the runnable interface

In Java, we can also create a thread by implementing the runnable interface. The runnable interface provides us both the run() method and the start() method.

Let's take an example to understand how we can create, start and run the thread using the Runnable interface.

ThreadExample2.java

```

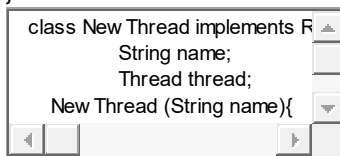
1. class NewThread implements Runnable {
2.     String name;
3.     Thread thread;
4.     NewThread (String name){
5.         this.name = name;
6.         thread = new Thread(this, name);
7.         System.out.println( "A New thread: " + thread+ "is created\n" );
8.         thread.start();
9.     }
10.    public void run() {
11.        try {
12.            for(int j = 5; j > 0; j--) {
13.                System.out.println(name + ": " + j);
14.                Thread.sleep(1000);
15.            }
16.        } catch (InterruptedException e) {
17.            System.out.println(name + " thread Interrupted");
18.        }
19.        System.out.println(name + " thread exiting.");
20.    }
21. }
22. class ThreadExample2 {
23.     public static void main(String args[]) {
24.         new NewThread("1st");
25.         new NewThread("2nd");
26.         new NewThread("3rd");
27.         try {
28.             Thread.sleep(8000);
29.         } catch (InterruptedException excetion) {
30.             System.out.println("Inturrption occurs in Main Thread");
31.         }
32.         System.out.println("We are exiting from Main Thread");

```

Nilam Suthar
Pgdca dept

33. }

34. }



Output:

```

C:\Users\ajeet\OneDrive\Desktop\programs>javac ThreadExample2.java

C:\Users\ajeet\OneDrive\Desktop\programs>java ThreadExample2
A New thread: Thread[1st,5,main]is created

A New thread: Thread[2nd,5,main]is created

A New thread: Thread[3rd,5,main]is created

3rd: 5
1st: 5
2nd: 5
3rd: 4
2nd: 4
1st: 4
1st: 3
3rd: 3
2nd: 3
1st: 2
3rd: 2
2nd: 2
3rd: 1
1st: 1
2nd: 1
3rd thread exiting.
1st thread exiting.
2nd thread exiting.
We are exiting from Main Thread

C:\Users\ajeet\OneDrive\Desktop\programs>_

```

In the above example, we perform the Multithreading by implementing the runnable interface. To learn

In Java, a thread always exists in any one of the following states. These states are:

1. New
2. Active

Nilam Suthar
Pgdc dept

3. Blocked / Waiting
4. Timed Waiting
5. Terminated

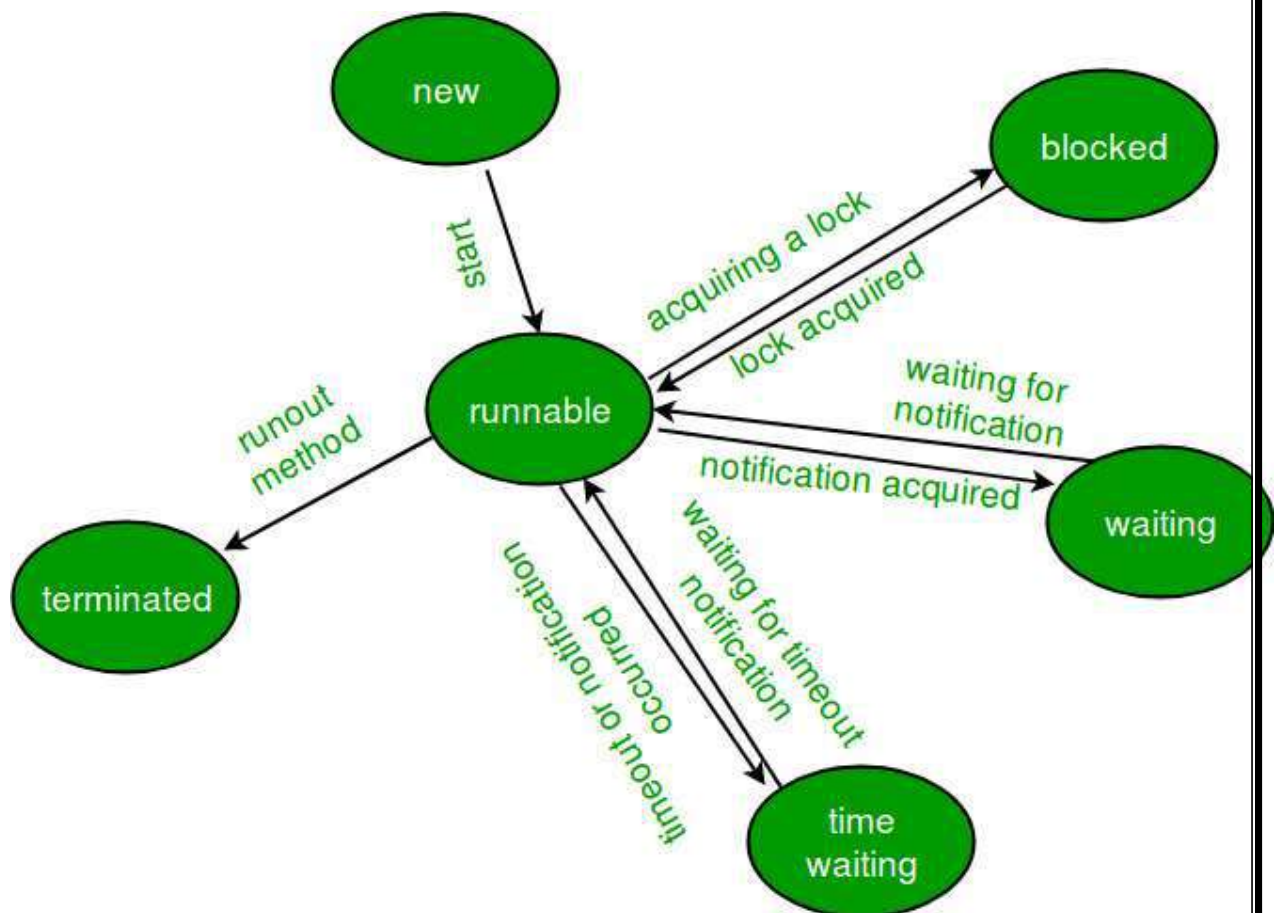
Lifecycle and States of a Thread in Java

A **thread** in Java at any point of time exists in any one of the following states.

A thread lies only in one of the shown states at any instant:

1. New
2. Runnable
3. Blocked
4. Waiting
5. Timed Waiting
6. Terminated

The diagram shown below represents various states of a thread at any instant in time.



Life Cycle of a thread

1. **New Thread:** When a new thread is created, it is in the new state.

2. The thread has not yet started to run when the thread is in this state.
3. When a thread lies in the new state, its code is yet to be run and hasn't started to execute.
4. **Runnable State:** A thread that is ready to run is moved to a runnable state.
5. In this state, a thread might actually be running or it might be ready to run at any instant of time.
6. It is the responsibility of the thread scheduler to give the thread, time to run. A multi-threaded program allocates a fixed amount of time to each individual thread.
7. Each and every thread runs for a short while and then pauses and relinquishes the CPU to another thread so that other threads can get a chance to run.
8. When this happens, all such threads that are ready to run, waiting for the CPU and the currently running thread lie in a runnable state.
9. **Blocked/Waiting state:** When a thread is temporarily inactive, then it's in one of the following states:
 - Blocked
 - Waiting
10. **Timed Waiting:** A thread lies in a timed waiting state when it calls a method with a time-out parameter.
11. A thread lies in this state until the timeout is completed or until a notification is received.
12. For example, when a thread calls sleep or a conditional wait, it is moved to a timed waiting state.
13. **Terminated State:** A thread terminates because of either of the following reasons:
 - Because it exits normally. This happens when the code of the thread has been entirely executed by the program.
 - Because there occurred some unusual erroneous event, like segmentation fault or an unhandled exception.

Implementing the Thread Stat

Explanation of Different Thread States

New: Whenever a new thread is created, it is always in the new state. For a thread in the new state, the code has not been run yet and thus has not begun its execution.

Active: When a thread invokes the start() method, it moves from the new state to the active state.

The active state contains two states within it: one is **runnable**, and the other is **running**.

- **Runnable:** A thread, that is ready to run is then moved to the runnable state.

- In the runnable state, the thread may be running or may be ready to run at any given instant of time.
- It is the duty of the thread scheduler to provide the thread time to run, i.e., moving the thread the running state. A program implementing multithreading acquires a fixed slice of time to each individual thread. Each and every thread runs for a short span of time and when that allocated time slice is over, the thread voluntarily gives up the CPU to the other thread, so that the other threads can also run for their slice of time.
- Whenever such a scenario occurs, all those threads that are willing to run, waiting for their turn to run, lie in the runnable state.
- In the runnable state, there is a queue where the threads lie.
- **Running:** When the thread gets the CPU, it moves from the runnable to the running state.
- Generally, the most common change in the state of a thread is from runnable to running and again back to runnable.

Blocked or Waiting: Whenever a thread is inactive for a span of time (not permanently) then, either the thread is in the blocked state or is in the waiting state.

```
import java.io.*;
class GFG extends Thread {
    public void run()
    {
        System.out.print("Welcome to GeeksforGeeks.");
    }
    public static void main(String[] args)
    {
        GFG g = new GFG(); // creating thread
        g.start(); // starting thread
    }
}
```

Multithreading in Java

Multithreading in Java is a process of executing multiple threads simultaneously.

A thread is a lightweight sub-process, the smallest unit of processing.

Multiprocessing and multithreading, both are used to achieve multitasking.

1. Extending the Thread class
2. Implementing the Runnable Interface
3. **Thread creation by extending the Thread class**
We create a class that extends the **java.lang.Thread** class.
4. This class overrides the run() method available in the Thread class.
5. A thread begins its life inside run() method.
6. We create an object of our new class and call start() method to start the execution of a thread. Start() invokes the run() method on the Thread object.

The **yield()** method causes the executing thread to stop temporarily and give the thread scheduler a chance to let another thread run.

If there is nothing else to run, the thread will continue executing without a context swap. The following example shows some of these thread-control methods at work. In this example, the main thread creates two threads and then waits for the first thread to finish by calling the first thread's **join()** method.

The first thread calls the **sleep()** method to be asleep for 10 seconds.

Meanwhile, the second thread calls its own **wait()** method to suspend itself until the main thread calls the **notify()** method of the suspended thread.

After the first thread comes to an end, the main thread method, and waits until the second thread comes to an end as well by calling the second thread's **join()** method.

sleep(long milliseconds) and sleep(long milliseconds, long nanoseconds)

These two methods cause the runtime environment to put the currently executing thread to sleep for the specified period.

This call must be placed within a **try-catch** block because an **InterruptedException** may be thrown if the thread is interrupted with **interrupt()**.

The thread does not automatically start after the specified sleep period, but it will be runnable.

Using the nanoseconds version of this method tends to be overkill, but it is available.

Stop()

Java Thread stop () method The stop () method of thread class terminates the thread execution. Once a thread is stopped, it cannot be restarted by start () method.

7. Priority of a Thread (Thread Priority)

Priorities are represented by a number between 1 and 10. In most cases,

the thread scheduler schedules the threads according to their priority (known as preemptive scheduling).

Setter & Getter Method of Thread Priority

public final int getPriority(): The java.lang.Thread.getPriority() method returns the priority of the given thread.

public final void setPriority(int newPriority): The java.lang.Thread.setPriority() method updates or assign the priority of the thread to newPriority.

The method throws IllegalArgumentException if the value newPriority goes out of the range, which is 1 (minimum) to 10 (maximum).

3 constants defined in Thread class:

1. public static int MIN_PRIORITY
2. public static int NORM_PRIORITY
3. public static int MAX_PRIORITY

Default priority of a thread is 5 (NORM_PRIORITY).

The value of MIN_PRIORITY is 1 and the value of MAX_PRIORITY is 10.

Example of priority of a Thread:

FileName: ThreadPriorityExample.java

```
1. // Importing the required classes
2. import java.lang.*;
3.
4. public class ThreadPriorityExample extends Thread
5. {
6.
7. // Method 1
8. // Whenever the start() method is called by a thread
9. // the run() method is invoked
10. public void run()
11. {
12. // the print statement
13. System.out.println("Inside the run() method");
14. }
15.
16. // the main method
17. public static void main(String args[])
18. {
19. // Creating threads with the help of ThreadPriorityExample class
20. ThreadPriorityExample th1 = new ThreadPriorityExample();
21. ThreadPriorityExample th2 = new ThreadPriorityExample();
22. ThreadPriorityExample th3 = new ThreadPriorityExample();
23.
24. // We did not mention the priority of the thread.
25. // Therefore, the priorities of the thread is 5, the default value
26.
27. // 1st Thread
28. // Displaying the priority of the thread
29. // using the getPriority() method
30. System.out.println("Priority of the thread th1 is : " + th1.getPriority());
31.
32. // 2nd Thread
33. // Display the priority of the thread
34. System.out.println("Priority of the thread th2 is : " + th2.getPriority());
```

Nilam Suthar
Pgdca dept

```
35.
    // 3rd Thread

1. // 3rd Thread
2. // // Display the priority of the thread
3. System.out.println("Priority of the thread th2 is : " + th2.getPriority());
4.
5. // Setting priorities of above threads by
6. // passing integer arguments
7. th1.setPriority(6);
8. th2.setPriority(3);
9. th3.setPriority(9);
10.
11. // 6
12. System.out.println("Priority of the thread th1 is : " + th1.getPriority());
13.
14. // 3
15. System.out.println("Priority of the thread th2 is : " + th2.getPriority());
16.
17. // 9
18. System.out.println("Priority of the thread th3 is : " + th3.getPriority());
19.
20. // Main thread
21.
22. // Displaying name of the currently executing thread
23. System.out.println("Currently Executing The Thread : " + Thread.currentThread().getName());
24.
25. System.out.println("Priority of the main thread is : " + Thread.currentThread().getPriority());
26.
27. // Priority of the main thread is 10 now
28. Thread.currentThread().setPriority(10);
29.
30. System.out.println("Priority of the main thread is : " + Thread.currentThread().getPriority());
```

31.}

32.}

Output:

```
Priority of the thread th1 is : 5
Priority of the thread th2 is : 5
Priority of the thread th2 is : 5
Priority of the thread th1 is : 6
Priority of the thread th2 is : 3
Priority of the thread th3 is : 9
Currently Executing The Thread : main
Priority of the main thread is : 5
Priority of the main thread is : 10
```

```
class MyThread extends Thread
{

    public void run()

    {

        System.out.println("Thread Running...");

    }

    public static void main(String[]args)

    {

        MyThread p1 = new MyThread();

        MyThread p2 = new MyThread();

        MyThread p3 = new MyThread();
```

Nilam Suthar
Pgdca dept

```
p1.start();

System.out.println("P1 thread priority : " + p1.getPriority());

System.out.println("P2 thread priority : " + p2.getPriority());

System.out.println("P3 thread priority : " + p3.getPriority());

    }
}
```

Copy

```
P1 thread priority : 5
Thread Running...
P2 thread priority : 5
P3 thread priority : 5
```

Example: Thread Constants

We can fetch priority of a thread by using some predefined constants provided by the Thread class.

these constants returns the max, min and normal priority of a thread.

```
class MyThread extends Thread
{

    public void run()

    {
```

```
        System.out.println("Thread Running...");

    }

    public static void main(String[]args)

    {

        MyThread p1 = new MyThread();

        p1.start();

        System.out.println("max thread priority : " + p1.MAX_PRIORITY);

        System.out.println("min thread priority : " + p1.MIN_PRIORITY);

        System.out.println("normal thread priority : " + p1.NORM_PRIORITY);

    }

}
```

Copy

```
Thread Running...
max thread priority : 10
min thread priority : 1
normal thread priority : 5
```

Example : Set Priority

To set priority of a thread, `setPriority()` method of thread class is used. It takes an integer argument that must be between 1 and 10. see the below example.

Nilam Suthar
Pgdc dept

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("Thread Running...");
    }

    public static void main(String[]args)
    {
        MyThread p1 = new MyThread();

        // Starting thread

        p1.start();

        // Setting priority

        p1.setPriority(2);

        // Getting priority

        int p = p1.getPriority();

        System.out.println("thread priority : " + p);

    }
}
```

}

Copy

thread priority : 2

Thread Running...

```

import java.lang.*;
public class Demo extends Thread{
    public void run(){
        System.out.println("Now, inside the run method");
    }
    public static void main(String[] args){
        Demo my_thr_1 = new Demo();
        Demo my_thr_2 = new Demo();
        System.out.println("The thread priority of first thread is : " +
my_thr_1.getPriority());
        System.out.println("The thread priority of first thread is : "
+
        my_thr_2.getPriority());
        my_thr_1.setPriority(5);
        my_thr_2.setPriority(3);
        System.out.println("The thread priority of first thread is : "
+
        my_thr_1.getPriority());
        System.out.println("The thread priority of first thread is : " +
my_thr_2.getPriority());
        System.out.print(Thread.currentThread().getName());
        System.out.println("The thread priority of main thread is : " +
Thread.currentThread().getPriority());
        Thread.currentThread().setPriority(10);
        System.out.println("The thread priority of main thread is : " +
Thread.currentThread().getPriority());
    }
}

```

Output

```

The thread priority of first thread is : 5
The thread priority of first thread is : 5
The thread priority of first thread is : 5
The thread priority of first thread is : 3
The thread priority of main thread is : 5
The thread priority of main thread is : 10

```

A class named Demo inherits from the base class Thread.

The function 'run' is defined and a relevant message is defined.

In the main function, two instances of the Demo class are created and their priorities are found by calling the function 'getPriority'.

They are printed on the console. Next, the Demo instances are assigned priority by using the 'setPriority' function.

The output is displayed on the console.

The names of the threads are printed on the screen with the help of the 'getName' function